

Python'da 1D Numpy

Maqsadlar

Ushbu amaliyotni tugatgandan so'ng, siz quyidagilarni bajara olasiz:

- `numpy` kutubxonasini chaqirish (import) va ishlatish
- `numpy` yordamida amallarni bajarish

Mundarija

- Numpy nima?
 - Turi (Type)
 - Qiymat berish
 - Qirqish (Slicing)
 - Ro'yxat yordamida qiymat berish
 - Boshqa atributlar
- Numpy massivlari ustida amallar
 - Massivlarni qo'shish
 - Massivlarni ko'paytirish
 - Ikki Numpy massivining ko'paytmasi
 - Nuqtali ko'paytma (Dot Product)
 - Numpy massiviga konstanta qo'shish
- Matematik funksiyalar
- Linspace

Numpy nima?

NumPy — bu massivlar, chiziqli algebra, Fure almashtirishlari va matritsalar bilan ishlash uchun mo'ljallangan Python kutubxonasi. NumPy «Numerical Python» (Raqamli Python) degan ma'noni anglatadi va u ochiq manbali loyihadir. NumPy ichidagi massiv obyekti **ndarray** deb ataladi va u ndarray bilan ishlashni osonlashtiruvchi ko'plab yordamchi funksiyalarni taqdim etadi.

Massivlar ma'lumotlar fani sohasida juda tez-tez ishlatiladi, chunki bu sohada tezlik va resurslar juda muhim ahamiyatga ega.

NumPy odatda `np` taxallusi (alias) ostida chaqiriladi.

U odatda o'zgarma o'lchamga ega bo'ladi va uning har bir elementi bir xil turdagi ma'lumotdan iborat bo'ladi. Biz avval `numpy` ni chaqirib, so'ngra ro'yxatni (list) numpy massiviga aylantirishimiz mumkin:

```
# numpy kutubxonasini chaqiramiz

import numpy as np
```

So'ngra ro'yxatni quyidagicha massivga aylantiramiz:

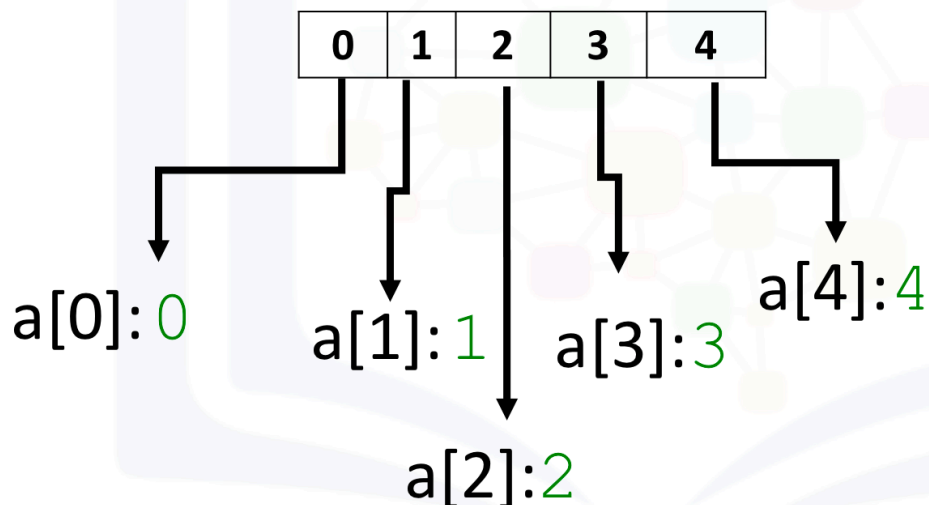
```
# numpy massivini yaratamiz

a = np.array([0, 1, 2, 3, 4])
a
```

```
array([0, 1, 2, 3, 4])
```

Har bir element bir xil ma'lumot turida bo'ladi, bu misolda butun sonlar (integers):

`a=np.array( [0, 1, 2, 3, 4] )`



Ro'yxatlardagi kabi, kvadrat qavslar orqali har bir elementga murojaat qilishimiz mumkin:

```
# Har bir elementni ekranga chiqaramiz

print("a[0]:", a[0])
print("a[1]:", a[1])
print("a[2]:", a[2])
print("a[3]:", a[3])
print("a[4]:", a[4])
```

```
a[0]: 0
a[1]: 1
a[2]: 2
a[3]: 3
a[4]: 4
```

NumPy versiyasini tekshirish

Versiya haqidagi ma'lumot qatori `__version__` atributi ostida saqlanadi.

```
print(np.__version__)

2.4.4
```

Turi (Type)

Agar massiv turini tekshirsak, **numpy.ndarray** natijasini olamiz:

```
# Massiv turini tekshiramiz

type(a)

numpy.ndarray
```

Numpy massivlari bir xil turdagi ma'lumotlarni saqlagani uchun, biz massiv elementlarining ma'lumot turini bilish maqsadida `dtype` atributidan foydalanishimiz mumkin. Ushbu misolda u 64-bitli butun son (integer):

```
# numpy massivida saqlangan qiymatlar turini tekshiramiz

a.dtype

dtype('int64')
```

O'zingiz sinab ko'ring

Berilgan **b** massivi uchun massivning turini va qiymatlar turini tekshiring.

```
b = np.array([3.1, 11.02, 6.2, 213.2, 5.2])

# Kodni shu yerga yozing
```

► Yechimni ko'rish uchun bu yerni bosing

Qiymat berish

Biz massivning qiymatini o'zgartirishimiz mumkin. `c` massivini ko'rib chiqamiz:

```
# Numpy massivini yaratamiz
```

```
c = np.array([20, 1, 2, 3, 4])  
c
```

```
array([20, 1, 2, 3, 4])
```

Biz massivning birinchi elementini quyidagicha 100 ga o'zgartirishimiz mumkin:

```
# Birinchi elementni 100 ga o'zgartiramiz  
  
c[0] = 100  
c
```

```
array([100, 1, 2, 3, 4])
```

Biz massivning 5-elementini quyidagicha 0 ga o'zgartirishimiz mumkin:

```
# 5-elementni 0 ga o'zgartiramiz  
  
c[4] = 0  
c
```

```
array([100, 1, 2, 3, 0])
```

O'zingiz sinab ko'ring

Berilgan massivning ikkinchi elementiga 20 qiymatini bering.

```
a = np.array([10, 2, 30, 40, 50])  
  
# Kodingizni shu yerga yozing
```

► Yechimni ko'rish uchun bu yerni bosing

Kesish

Ro'yxatlardagi kabi, biz numpy massivini ham kesishimiz (slice) mumkin. Pythonda kesish bu ma'lum bir indeksdan boshqa bir berilgan indeks gacha bo'lgan elementlarni olishni anglatadi.

Biz kesishni quyidagicha yozamiz: `[start:end]`. Oxirgi indeksdagi (end) element natijaga kiritilmaydi.

Biz 1-dan 3-gacha bo'lgan elementlarni tanlab, uni yangi `d` numpy massiviga quyidagicha biriktirishimiz mumkin:

```
# Numpy massivini kesish  
  
d = c[1:4]  
d
```

```
array([1, 2, 3])
```

Biz mos keladigan indeksdarga yangi qiymatlarni quyidagicha berishimiz mumkin:

```
# To'rtinchi va beshinchi elementlarga 300 va 400 qiymatlarini beramiz
```

```
c[3:5] = 300, 400
```

```
c
```

```
array([100, 1, 2, 300, 400])
```

Biz kesishda qadamlarni ham quyidagicha belgilashimiz mumkin: `[start:end:step]`.

```
arr = np.array([1, 2, 3, 4, 5, 6, 7])
```

```
print(arr[1:5:2])
```

```
[2 4]
```

Agar biz boshlang'ich nuqtani (start) bermasak, u 0 deb hisoblanadi.

```
print(arr[:4])
```

```
[1 2 3 4]
```

Agar biz oxirgi nuqtani (end) bermasak, u massivning oxirigacha (massiv uzunligigacha) deb hisoblanadi.

```
print(arr[4:])
```

```
[5 6 7]
```

Agar biz qadamni (step) bermasak, u 1 deb hisoblanadi.

```
print(arr[1:5:])
```

```
[2 3 4 5]
```

O'zingiz sinab ko'ring

Berilgan massivdagi juft elementlarni chop eting.

```
arr = np.array([1, 2, 3, 4, 5, 6, 7, 8])
```

```
# Kodingizni shu yerga yozing
```

► Yechimni ko'rish uchun bu yerni bosing

Qiymatni ro'yxat bilan birlashtirish

Shuningdek, biz bir nechta aniq indeksni tanlash uchun ro'yxatdan foydalanishimiz mumkin. `select` ro'yxati bir nechta qiymatlarni o'z ichiga oladi:

```
# Indekslar ro'yxatini yaratish
```

```
select = [0, 2, 3, 4]
select
```

```
[0, 2, 3, 4]
```

Biz ro'yxatni qavslar ichida argument sifatida ishlatishimiz mumkin. Natija esa ushbu aniq indekslarga mos keladigan elementlardir:

```
# Elementlarni tanlash uchun ro'yxatdan foydalanish
```

```
d = c[select]
d
```

```
array([100,    2, 300, 400])
```

Biz ko'rsatilgan elementlarga yangi qiymat berishimiz mumkin. Masalan, qiymatlarni quyidagicha 100 000 ga o'zgartirishimiz mumkin:

```
# Ko'rsatilgan elementlarga yangi qiymat berish
```

```
c[select] = 100000
c
```

```
array([100000,    1, 100000, 100000, 100000])
```

Boshqa atributlar

Keling, `a` massividan foydalangan holda massivning ba'zi asosiy atributlarini ko'rib chiqamiz:

```
# Numpy massivini yaratish
```

```
a = np.array([0, 1, 2, 3, 4])
a
```

```
array([0, 1, 2, 3, 4])
```

`size` atributi massivdagi elementlar sonini bildiradi:

```
# Numpy massivining o'lchamini olish
```

```
a.size
```

Keyingi ikkita atribut yuqori o'lchamlarga o'tganimizda yanada tushunarliroq bo'ladi, lekin hozircha ularni ko'rib chiqamiz. `ndim` atributi massiv o'lchamlari sonini yoki massiv rangini (rank) bildiradi. Bu holatda, u birga teng:

```
# Numpy massivining o'lchamlari sonini olish
```

```
a.ndim
```

```
1
```

`shape` atributi massivning har bir o'lchamdagi hajmini ko'rsatuvchi butun sonlar korteji (tuple) hisoblanadi:

```
# Numpy massivining shaklini (shape) olish
```

```
a.shape
```

```
(5,)
```

O'zingiz sinab ko'ring

Berilgan **b** massivi uchun o'lcham (size), o'lchamlar soni (dimension) va shaklni (shape) toping.

```
b = np.array([10, 20, 30, 40, 50, 60, 70])
```

```
# Kodni shu yerga yozing
```

► Yechimni ko'rish uchun bu yerni bosing

Numpy Statistik Funksiyalari

```
# Numpy massivini yaratish
```

```
a = np.array([1, -1, 1, -1])
```

```
# Numpy massivining o'rtacha qiymatini olish
```

```
mean = a.mean()
```

```
mean
```

```
np.float64(0.0)
```

```
# Numpy massivining standart og'ishini olish
```

```
standard_deviation=a.std()
```

```
standard_deviation
```

```
np.float64(1.0)
```

```
# Numpy massivini yaratish
```

```
b = np.array([-1, 2, 3, 4, 5])  
b
```

```
array([-1,  2,  3,  4,  5])
```

```
# Numpy massividagi eng katta qiymatni olish
```

```
max_b = b.max()  
max_b
```

```
np.int64(5)
```

```
# Numpy massividagi eng kichik qiymatni olish
```

```
min_b = b.min()  
min_b
```

```
np.int64(-1)
```

O'zingiz sinab ko'ring

Berilgan numpy massividagi eng katta va eng kichik qiymatlar yig'indisini toping

```
c = np.array([-10, 201, 43, 94, 502])
```

```
# Kodni shu yerga yozing
```

► Yechimni ko'rish uchun bu yerni bosing

Numpy massiv operatsiyalari

Arifmetik operatorlardan to'g'ridan-to'g'ri NumPy massivlari orasida foydalanishingiz mumkin

Massivlarni qo'shish

Numpy massivi `u` ni ko'rib chiqamiz:

```
u = np.array([1, 0])  
u
```

```
array([1, 0])
```

Numpy massivi `v` ni ko'rib chiqamiz:

```
v = np.array([0, 1])  
v
```



```
array([0, 1])
```

Biz ikkita massivni qo'shib, natijani `z` ga o'zlashtirishimiz mumkin:

```
# Numpy massivlarini qo'shish
```

```
z = np.add(u, v)
z
```

```
array([1, 1])
```

Ushbu operatsiya vektorlarni qo'shishga ekvivalentdir:

```
# Grafik chizish funksiyalari
```

```
import time
```

```
import sys
```

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

```
def Plotvec1(u, z, v):
```

```
    ax = plt.axes() # To'liq oyna o'qlarini yaratish
```

```
    ax.arrow(0, 0, *u, head_width=0.05, color='r', head_length=0.1)
```

```
    # U o'qiga uchi 0.05 kenglikdagi, qizil rangli va 0.1 uzunlikdagi st
    plt.text(*(u + 0.1), 'u') # O'qlarga 'u' matnini qo'shish
```

```
    ax.arrow(0, 0, *v, head_width=0.05, color='b', head_length=0.1)
```

```
    # V o'qiga uchi 0.05 kenglikdagi, ko'k rangli va 0.1 uzunlikdagi str
    plt.text(*(v + 0.1), 'v') # O'qlarga 'v' matnini qo'shish
```

```
    ax.arrow(0, 0, *z, head_width=0.05, head_length=0.1)
```

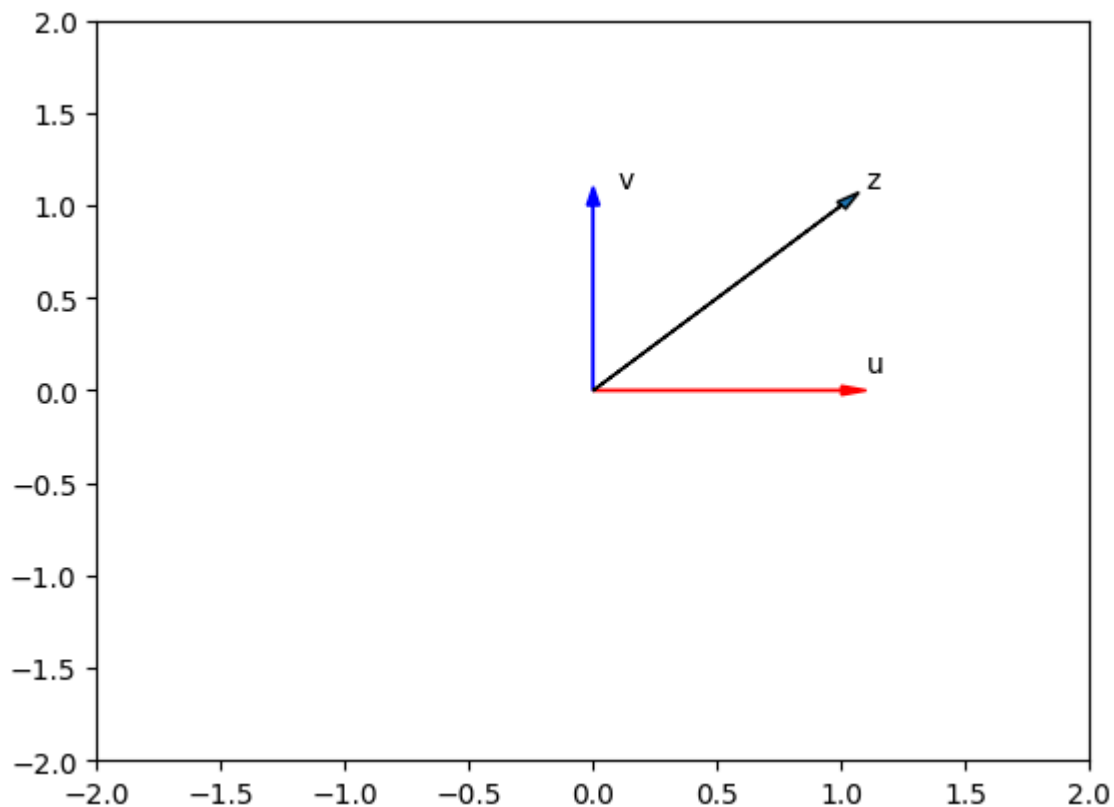
```
    plt.text(*(z + 0.1), 'z') # O'qlarga 'z' matnini qo'shish
```

```
    plt.ylim(-2, 2) # Y o'qi chegarasini pastki (-2) va yuqori (2) qilib
```

```
    plt.xlim(-2, 2) # X o'qi chegarasini chap (-2) va o'ng (2) qilib bel
```

```
# Numpy massivlarini grafikda chizish
```

```
Plotvec1(u, z, v)
```



O'zingiz sinab ko'ring

Berilgan arr1 va arr2 numpy massivlari ustida qo'shish amalini bajaring:

```
arr1 = np.array([10, 11, 12, 13, 14, 15])
arr2 = np.array([20, 21, 22, 23, 24, 25])
```

Kodni shu yerga yozing

► Yechimni ko'rish uchun bu yerni bosing

Numpy massivi **a** ni ko'rib chiqamiz:

```
a = np.array([10, 20, 30])
a
```

```
array([10, 20, 30])
```

Numpy massivi **b** ni ko'rib chiqamiz:

```
b = np.array([5, 10, 15])
b
```

```
array([ 5, 10, 15])
```

Biz ikkita massivni ayirishimiz va natijani **c** ga o'zlashtirishimiz mumkin:

```
c = np.subtract(a, b)
print(c)
```

[5 10 15]

O'zingiz sinab ko'ring

Berilgan arr1 va arr2 numpy massivlari ustida ayirish amalini bajaring:

```
arr1 = np.array([10, 20, 30, 40, 50, 60])
arr2 = np.array([20, 21, 22, 23, 24, 25])
```

Kodni shu yerga yozing

► Yechimni ko'rish uchun bu yerni bosing

Massivlarni ko'paytirish

Numpy vektor massivi `y` ni ko'rib chiqing:

Numpy massivini yaratish

```
x = np.array([1, 2])
x
```

array([1, 2])

Numpy massivini yaratish

```
y = np.array([2, 1])
y
```

array([2, 1])

Biz massivning har bir elementini 2 ga ko'paytirishimiz mumkin:

Numpy massivlarini ko'paytirish

```
z = np.multiply(x, y)
z
```

array([2, 2])

Bu skalyar ko'paytirishga, ya'ni vektorni songa ko'paytirishga teng:

O'zingiz sinab ko'ring

Berilgan arr1 va arr2 numpy massivlari ustida ko'paytirish amalini bajaring:

```
arr1 = np.array([10, 20, 30, 40, 50, 60])
arr2 = np.array([2, 1, 2, 3, 4, 5])
```

Kodni shu yerga yozing

► Yechimni ko‘rish uchun bu yerni bosing

Massivlarni bo'lish

Numpy vektor massivi `a` ni ko'rib chiqing:

```
a = np.array([10, 20, 30])  
a
```

```
array([10, 20, 30])
```

Numpy vektor massivi `b` ni ko'rib chiqing:

```
b = np.array([2, 10, 5])  
b
```

```
array([ 2, 10,  5])
```

Biz ikkita massivni bo'lishimiz va natijani `c` ga o'zlashtirishimiz mumkin:

```
c = np.divide(a, b)  
c
```

```
array([5.,  2.,  6.])
```

O‘zingiz sinab ko‘ring

Berilgan `arr1` va `arr2` numpy massivlari ustida bo‘lish amalini bajaring:

```
arr1 = np.array([10, 20, 30, 40, 50, 60])  
arr2 = np.array([3, 5, 10, 8, 2, 33])
```

```
# Kodni shu yerga yozing
```

► Yechimni ko‘rish uchun bu yerni bosing

Skalyar ko'paytma

Ikki numpy massiv `u` va `v` ning skalyar ko‘paytmasi quyidagicha aniqlanadi:

```
X = np.array([1, 2])  
Y = np.array([3, 2])
```

```
# Skalyar ko'paytmani hisoblash
```

```
np.dot(X, Y)
```

```
np.int64(7)
```

```
# X ning elementlari  
print(X[0])
```

```
print(X[1])
```

1

2

```
# Y ning elementlari
```

```
print(Y[0])
```

```
print(Y[1])
```

3

2

Biz quyida ko'rsatilganidek skalyar ko'paytmani bajaryapmiz:

$$\begin{array}{ccc} \text{Elements of X} & & \text{Elements of Y} \\ x[0] & x[1] & y[0] \quad y[1] \\ X = [1, & 2] & Y = [3, & 2] \\ \\ X.Y = [(X[0] * Y[0]) + (X[1] * Y[1])] \\ = [(1 * 3) + (2 * 2)] \\ = [3 + 4] \\ = 7 \end{array}$$

O'zingiz sinab ko'ring

Berilgan arr1 va arr2 numpy massivlari ustida skalyar ko'paytirish (dot) amalini bajaring:

```
arr1 = np.array([3, 5])
```

```
arr2 = np.array([2, 4])
```

```
# Kodni shu yerga yozing
```

► Yechimni ko'rish uchun bu yerni bosing

Numpy massiviga o'zgarmas son qo'shish

Quyidagi massivni ko'rib chiqing:

```
# Numpy massiviga o'zgarmas son (constant) qo'shish
```

```
u = np.array([1, 2, 3, -1])
```

```
u
```

```
array([ 1,  2,  3, -1])
```

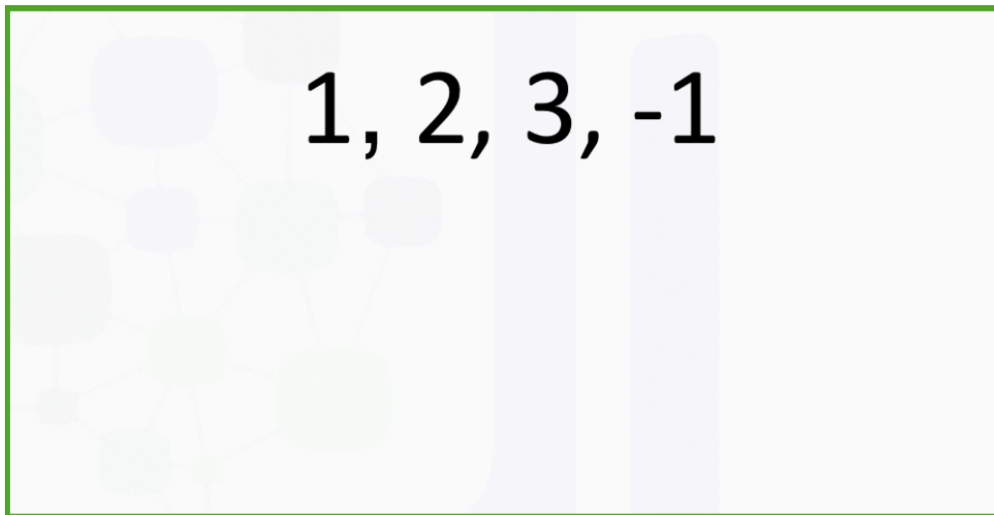
Massivning har bir elementiga 1 o'zgarmas sonini qo'shish:

```
# Massivga o'zgarmas sonni qo'shish
```

```
u + 1
```

```
array([2, 3, 4, 0])
```

Jarayon quyidagi animatsiyada umumlashtirilgan:



O'zingiz sinab ko'ring

Berilgan `arr` numpy massivining har bir elementiga 5 o'zgarmas sonini qo'shing:

```
arr = np.array([1, 2, 3, -1])
```

```
# Kodni shu yerga yozing
```

► Yechimni ko'rish uchun bu yerni bosing

Matematik funksiyalar

Numpy-da `pi` qiymatini quyidagicha olishimiz mumkin:

```
# pi qiymati
```

```
np.pi
```

```
3.141592653589793
```

Biz quyidagi numpy massivini radianlarda yaratishimiz mumkin:

```
# Radianlarda numpy massivini yaratish
```

```
x = np.array([0, np.pi/2, np.pi])
```

Biz `sin` funksiyasini `x` massiviga qo'llashimiz va qiymatlarni `y` massiviga o'zlashtirishimiz mumkin; bu massivning har bir elementiga sinus funksiyasini qo'llaydi:

```
# Har bir elementning sinusini hisoblash
```

```
y = np.sin(x)  
y
```

```
array([0.0000000e+00, 1.0000000e+00, 1.2246468e-16])
```

Linspace

Matematik funksiyalarni chizish uchun foydali funksiya `linspace` hisoblanadi. `Linspace` belgilangan interval bo'yicha bir xil masofadagi sonlarni qaytaradi.

`numpy.linspace(start, stop, num = int value)`

start : interval oralg'ining boshi

stop : interval oralg'ining oxiri

num : hosil qilinadigan namunalar soni

```
# [-2, 2] oralg'ida 5 ta elementdan iborat numpy massivini yaratish
```

```
np.linspace(-2, 2, num=5)
```

```
array([-2., -1.,  0.,  1.,  2.])
```

Agar biz `num` parametrini 9 ga o'zgartirsak, -2 dan 2 gacha bo'lgan oraliqda bir xil masofada joylashgan 9 ta sonni olamiz:

```
# [-2, 2] oralg'ida 9 ta elementdan iborat numpy massivini yaratish
```

```
np.linspace(-2, 2, num=9)
```

```
array([-2. , -1.5, -1. , -0.5,  0. ,  0.5,  1. ,  1.5,  2. ])
```

Biz `linspace` funksiyasidan 0 dan 2π gacha bo'lgan oraliqda bir xil masofada joylashgan 100 ta namunani yaratish uchun foydalanishimiz mumkin:

```
# [0, 2π] oralg'ida 100 ta elementdan iborat numpy massivini yaratish
```

```
x = np.linspace(0, 2*np.pi, num=100)
```

Biz `sin` funksiyasini `x` massivining har bir elementiga qo'llashimiz va uni `y` massiviga o'zlashtirishimiz mumkin:

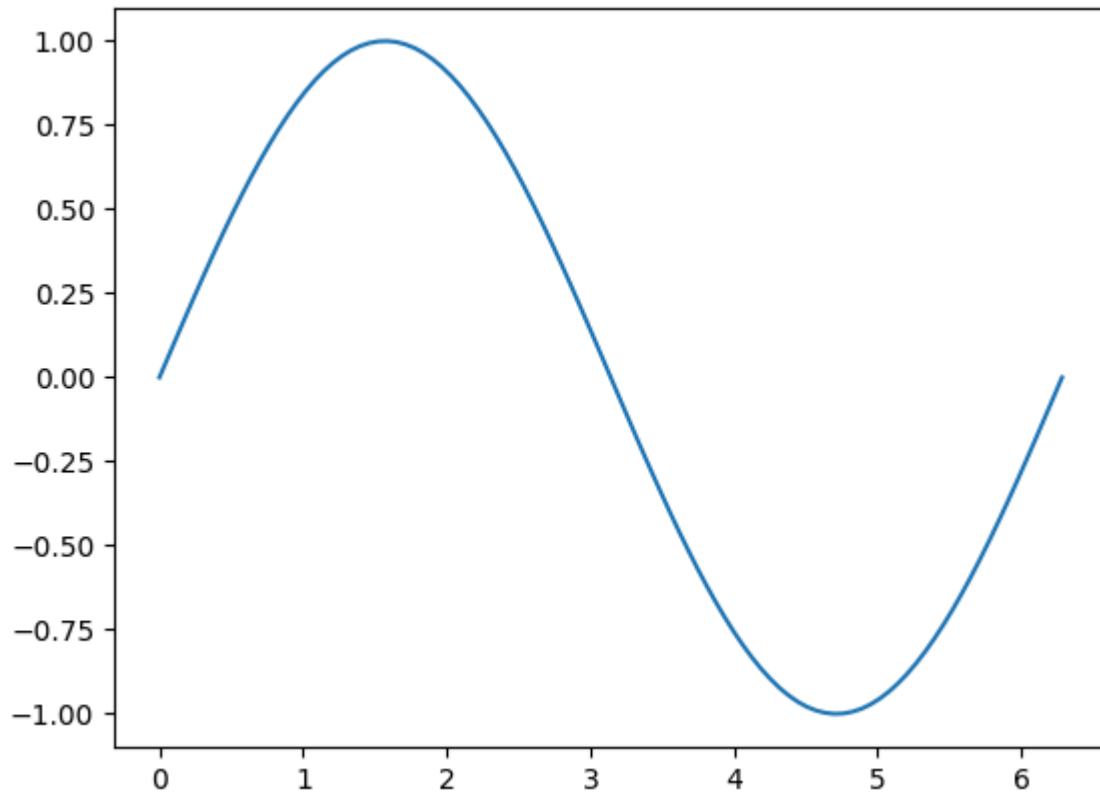
```
# x ro'yxatining sinusini hisoblash
```

```
y = np.sin(x)
```

```
# Natijani chizish
```

```
plt.plot(x, y)
```

```
[<matplotlib.lines.Line2D at 0x1da99dc4690>]
```



O'zingiz sinab ko'ring

[5, 4] oralig'ida 6 ta elementdan iborat numpy massivini yarating:

```
# Kodni shu yerga yozing
```

► Yechimni ko'rish uchun bu yerni bosing

1D massivlarni iteratsiya qilish (takrorlash)

Iteratsiya qilish — bu elementlarni birma-bir ko'rib chiqish deganidir.

Agar biz 1D (bir o'lchovli) massivni iteratsiya qilsak, u har bir element orqali tartib bilan o'tadi.

Numpy massivini bevosita chiqarganimizda, u massiv (array) formatida ko'rinadi:

```
arr1 = np.array([1, 2, 3])  
print(arr1)
```



```
[1 2 3]
```

Ammo natijani ro'yxat (list) shaklida yoki elementma-element olishni istasangiz, `for` siklidan foydalanishingiz mumkin:

```
for x in arr1:  
    print(x)
```

```
1  
2  
3
```

1D Numpy massivlari bo'yicha viktorina

Numpyda quyidagi vektor ayirmasini amalga oshiring: $u - v$

Pastga kodni yozing va bajarish uchun Shift+Enter tugmalarini bosing

```
u = np.array([1, 0])  
v = np.array([0, 1])
```

► Yechimni ko'rish uchun bu yerni bosing

z numpy massivini -2 ga ko'paytiring:

Kodni pastga yozing va bajarish uchun Shift+Enter tugmalarini bosing

```
z = np.array([2, 4])
```

► Yechimni ko'rish uchun bu yerni bosing

`[1, 2, 3, 4, 5]` va `[1, 0, 1, 0, 1]` ro'yxatlarini ko'rib chiqing. Ikkala ro'yxatni numpy massiviga o'tkazing, so'ngra ularni bir-biriga ko'paytiring:

Kodni pastga yozing va bajarish uchun Shift+Enter tugmalarini bosing

► Yechimni ko'rish uchun bu yerni bosing

Import the Libraries

```
import time  
import sys  
import numpy as np  
  
import matplotlib.pyplot as plt
```

```
def Plotvec2(a,b):  
    ax = plt.axes()# to generate the full window axes
```

```
ax.arrow(0, 0, *a, head_width=0.05, color='r', head_length=0.1)#Add
plt.text(*(a + 0.1), 'a')
ax.arrow(0, 0, *b, head_width=0.05, color='b', head_length=0.1)#Add
plt.text(*(b + 0.1), 'b')
plt.ylim(-2, 2)#set the ylim to bottom(-2), top(2)
plt.xlim(-2, 2)#set the xlim to left(-2), right(2)
```

`[-1, 1]` va `[1, 1]` ro'yxatlarini `a` va `b` numpy massivlariga o'tkazing. So'ngra, `Plotvec2` funksiyasi yordamida massivlarni vektorlar sifatida chizing va ularning skalyar ko'paytmasini (dot product) toping:

```
# Kodni pastga yozing va bajarish uchun Shift+Enter tugmalarini bosing
```

► Yechimni ko'rish uchun bu yerni bosing

`[1, 0]` va `[0, 1]` ro'yxatlarini `a` va `b` numpy massivlariga o'tkazing. So'ngra, `Plotvec2` funksiyasi yordamida massivlarni vektorlar sifatida chizing va ularning skalyar ko'paytmasini (dot product) toping:

```
# Kodingizni pastga yozing va bajarish uchun Shift+Enter tugmalarini bos
```

► Yechimni ko'rish uchun bu yerni bosing

`[1, 1]` va `[0, 1]` ro'yxatlarini `a` va `b` numpy massivlariga o'tkazing. So'ngra, `Plotvec2` funksiyasi yordamida massivlarni vektorlar sifatida chizing va ularning skalyar ko'paytmasini (dot product) toping:

```
# Kodni pastga yozing va bajarish uchun Shift+Enter tugmalarini bosing
```

► Yechimni ko'rish uchun bu yerni bosing

Nega `[-1, 1]` va `[1, 1]` vektorlarining skalyar ko'paytmasi hamda `[1, 0]` va `[0, 1]` vektorlarining skalyar ko'paytmasi natijalari nolga teng, ammo `[1, 1]` va `[0, 1]` uchun esa nol emas?

Yordam: Tegishli chizmalarni o'rganing, ko'rsatkichlar (strelkalar) qaysi yo'nalishga qarab turganiga e'tibor bering.

```
# Kodni pastga yozing va bajarish uchun Shift+Enter tugmalarini bosing
```

► Yechimni ko'rish uchun bu yerni bosing

`[1, 2, 3]` va `[8, 9, 10]` ro'yxatlarini `arr1` va `arr2` numpy massivlariga o'tkazing. So'ngra `arr1` va `arr2` ustida **Qo'shish** (Addition), **Ayirish** (Subtraction), **Ko'paytirish** (Multiplication), **Bo'lish** (Division) va **Skalyar ko'paytma** (Dot Operation) amallarini bajaring.

Kodni pastga yozing va bajarish uchun Shift+Enter tugmalarini bosing

► Yechimni ko‘rish uchun bu yerni bosing

[1, 2, 3, 4, 5] va [6, 7, 8, 9, 10] ro‘yxatlarini `arr1` va `arr2` numpy massivlariga o‘tkazing. So‘ngra `arr1` va `arr2` massivlaridan juft va toq sonlarni toping:

Kodni pastga yozing va bajarish uchun Shift+Enter tugmalarini bosing

► Yechimni ko‘rish uchun bu yerni bosing

Oxirgi mashq!

Tabriklaymiz, siz Python bo'yicha birinchi darsingizni va amaliy mashg'ulotingizni muvaffaqiyatli yakunladingiz.
